

# Visualizing Relationships between Communication Apprehension and Public Transit Usage

AUTHOR

Weizhu Pan

## Introduction

This memo evaluates different visual strategies for illustrating the potential relationship between communication apprehension and public transportation usage. Given the complex nature of the dependent variable—which can be operationalized as continuous midpoint estimates, ordinal categories, or binary user status—we compare four distinct visualization approaches to assess their strengths and limitations for different data types and analytical contexts.

## Data Preparation

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the data
df = pd.read_csv('../../original_data/merged_data.csv')
df = df.iloc[2:].reset_index(drop=True)

# Define the Likert scale mapping
likert_mapping = {
    "Strongly disagree": 1,
    "Somewhat disagree": 2,
    "Neither agree nor disagree": 3,
    "Somewhat agree": 4,
    "Strongly agree": 5
}

# Communication apprehension items to recode
communication_apprehension_items = ['Q1', 'Q2', 'Q3', 'Q4', 'Q5', 'Q6', 'Q13', 'Q14', 'Q15', 'Q16', 'Q17']

# Items to reverse-code (Q2, Q4, Q6, Q14, Q16, Q17)
reverse_code_items = ['Q2', 'Q4', 'Q6', 'Q14', 'Q16', 'Q17']

# Recode communication apprehension items
for item in communication_apprehension_items:
    if item in df.columns:
        df[item] = df[item].map(likert_mapping)

# Reverse-code items
```

```

for item in reverse_code_items:
    if item in df.columns:
        df[item] = 6 - df[item]

# Compute composite communication apprehension score
communication_apprehension_columns = [col for col in communication_apprehension_items if
df['communication_apprehension_score'] = df[communication_apprehension_columns].sum(axis=

# Recode public transit usage frequency (Q26) - midpoint estimation
transit_midpoint_mapping = {
    "Never": 0.0,
    "0-1 days a month": 0.5,
    "2-4 days a month": 3.0,
    "4-8 days a month": 6.0,
    "8 or more days a month": 12.0
}

if 'Q26' in df.columns:
    df['public_transit_days'] = df['Q26'].map(transit_midpoint_mapping)

# Binary indicator for transit usage
if 'Q26' in df.columns:
    df['is_transit_user'] = df['Q26'].apply(lambda x: 0 if x == "Never" else 1 if pd.notn

# Drop NA values
df_clean = df.dropna(subset=['communication_apprehension_score', 'public_transit_days', '

print(f"Sample size after cleaning: {len(df_clean)}")

```

Sample size after cleaning: 250

## Visualization Strategy 1: Box Plot

```

# Get original transit categories in order
transit_order = ["Never", "0-1 days a month", "2-4 days a month", "4-8 days a month", "8

# Filter to only include valid transit categories
df_plot = df_clean[df_clean['Q26'].isin(transit_order)].copy()
df_plot['Q26'] = pd.Categorical(df_plot['Q26'], categories=transit_order, ordered=True)

# Create box plot
plt.figure(figsize=(6.5, 4))
sns.boxplot(x='communication_apprehension_score', y='Q26', data=df_plot,
            order=transit_order, palette=['lightgray', 'lightblue', 'steelblue', 'darkblu
plt.xlabel('Communication Apprehension Score')
plt.ylabel('Public Transit Usage Frequency')
plt.title('Communication Apprehension Score by Transit Usage Category')
plt.grid(axis='x', alpha=0.3)

```

```
plt.tight_layout()
plt.show()
```

/var/folders/26/xd7zmrq533n6l0jgkczv\_w7w0000gn/T/ipykernel\_91237/4117204890.py:10:  
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.  
Assign the `y` variable to `hue` and set `legend=False` for the same effect.

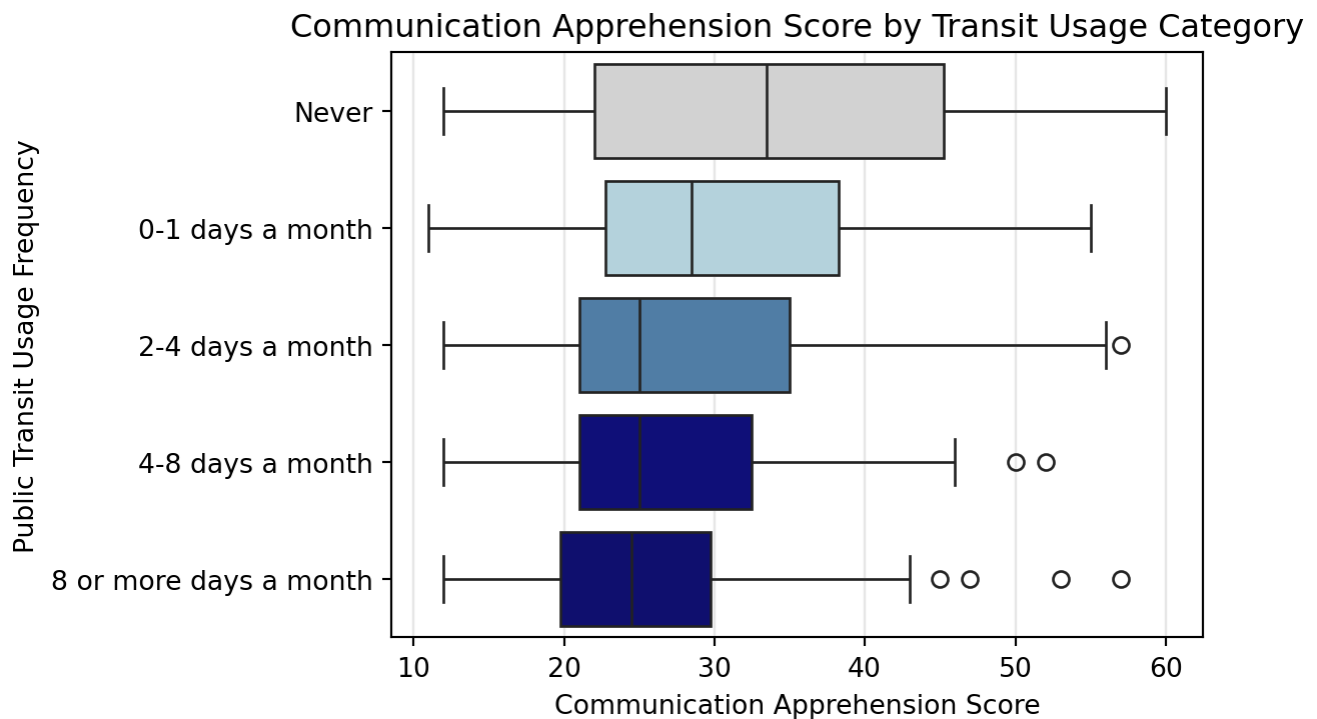


Figure 1: Box plot of communication apprehension score by transit usage categories

## Visualization Strategy 2: Jittered OLS Regression

```
import statsmodels.api as sm

# Prepare data for regression
X = df_clean['communication_apprehension_score']
X = sm.add_constant(X)
y = df_clean['public_transit_days']

# Run OLS regression
model_ols = sm.OLS(y, X).fit()

# Add jitter to points
np.random.seed(42)
jitter_y = df_clean['public_transit_days'] + np.random.normal(0, 0.3, len(df_clean))
```

```

jitter_x = df_clean['communication_apprehension_score'] + np.random.normal(0, 0.05, len(d

# Create plot
plt.figure(figsize=(6.5, 4.5))
plt.scatter(jitter_x, jitter_y, alpha=0.5, s=30, color='steelblue', label='Observed Data'

# Plot regression line with confidence interval
x_pred = np.linspace(df_clean['communication_apprehension_score'].min(), df_clean['commun
x_pred_const = sm.add_constant(x_pred)
y_pred = model_ols.predict(x_pred_const)
y_pred_ci = model_ols.get_prediction(x_pred_const).conf_int()

plt.plot(x_pred, y_pred, 'r-', linewidth=2, label='OLS Regression Line')
plt.fill_between(x_pred, y_pred_ci[:, 0], y_pred_ci[:, 1], alpha=0.2, color='red', label=

plt.xlabel('Communication Apprehension Score')
plt.ylabel('Public Transit Days in Last 3 Months (Midpoint)')
plt.title('OLS Regression with Jittered Points and Confidence Interval')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```

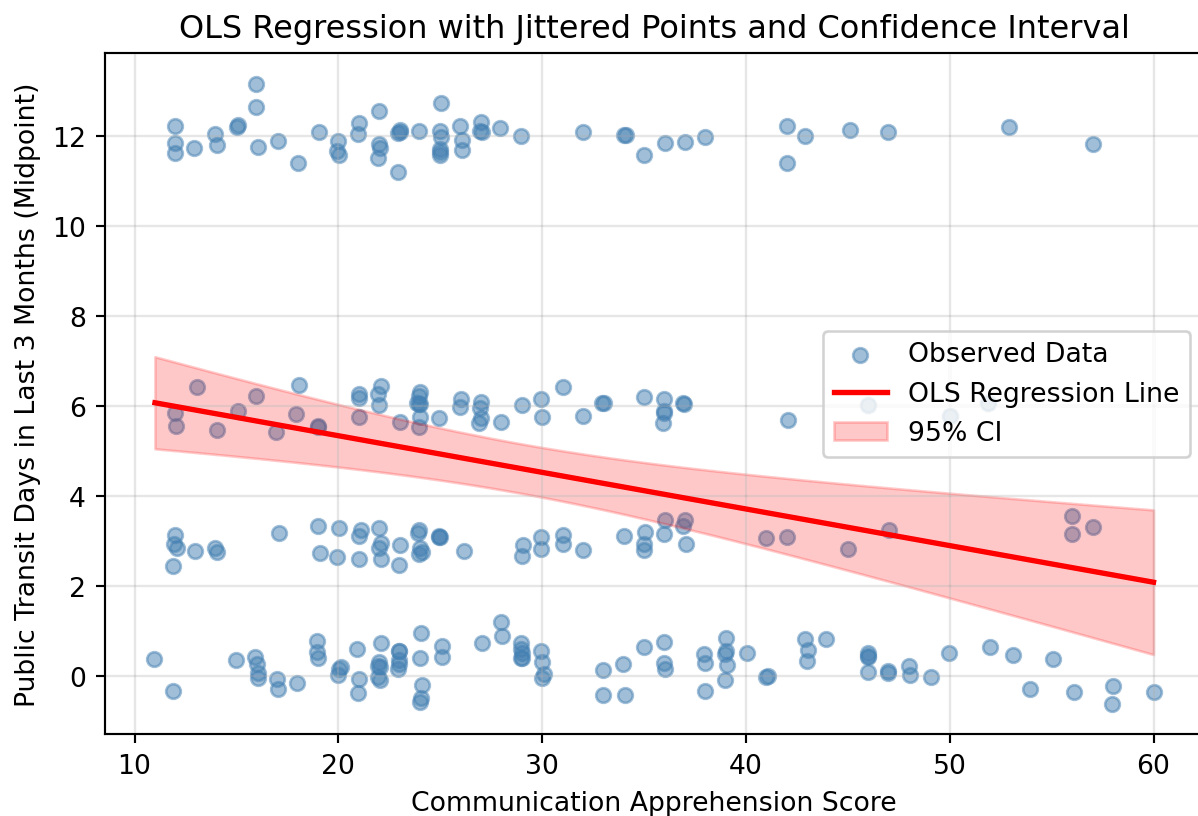


Figure 2: OLS regression plot with jittered points and 95% confidence interval

## Visualization Strategy 3: Logistic Regression Sigmoid

```

# Prepare data for logistic regression
X_logit = df_clean['communication_apprehension_score']
X_logit = sm.add_constant(X_logit)
y_logit = df_clean['is_transit_user']

# Run logistic regression
model_logit = sm.Logit(y_logit, X_logit).fit()

# Generate predicted probabilities across full range
x_range = np.linspace(df_clean['communication_apprehension_score'].min(),
                      df_clean['communication_apprehension_score'].max(), 100)
x_range_const = sm.add_constant(x_range)
predicted_probs = model_logit.predict(x_range_const)

# Add jitter to binary data points
np.random.seed(42)
jitter_y = df_clean['is_transit_user'] + np.random.normal(0, 0.03, len(df_clean))
jitter_x = df_clean['communication_apprehension_score'] + np.random.normal(0, 0.02, len(d

# Create plot with minimal styling
plt.figure(figsize=(6, 3.5))
plt.scatter(jitter_x, jitter_y, color='black', alpha=0.3, s=20, label='Observed Data')
plt.plot(x_range, predicted_probs, 'b-', linewidth=2, label='Fitted Probability')

# Styling
plt.xlabel('Communication Apprehension Score')
plt.ylabel('Probability of Being a Transit User')
plt.title('Logistic Regression: Predicted Probability of Transit User Status')
plt.ylim(-0.1, 1.1)
plt.yticks([0.00, 0.25, 0.50, 0.75, 1.00])
plt.xlim(df_clean['communication_apprehension_score'].min(),
         df_clean['communication_apprehension_score'].max())

# Remove top and right spines
ax = plt.gca()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_color('black')
ax.spines['bottom'].set_color('black')

# Minimal grid
ax.grid(axis='y', alpha=0.1, linestyle='--')

plt.legend()
plt.tight_layout()
plt.show()

```

Optimization terminated successfully.  
 Current function value: 0.475655  
 Iterations 6

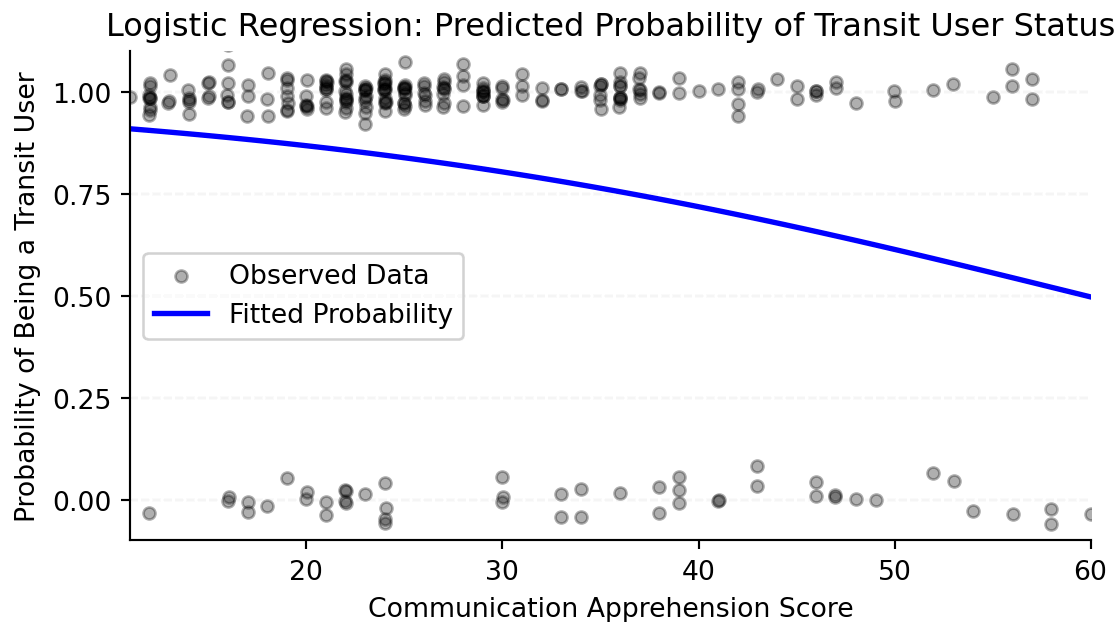


Figure 3: Logistic regression sigmoid curve with jittered binary points

## Visualization Strategy 4: Proportional Stacked Bar Chart

```
# Create quartiles for communication apprehension
df_clean['ca_quartile'] = pd.qcut(df_clean['communication_apprehension_score'], q=4,
                                  labels=['Low', 'Medium-Low', 'Medium-High', 'High'])

# Get original transit categories
transit_order = ["Never", "0-1 days a month", "2-4 days a month", "4-8 days a month", "8
days a month or more"]

# Create crosstab
crosstab = pd.crosstab(df_clean['ca_quartile'], df_clean['Q26'], normalize='index')
crosstab = crosstab[transit_order] # Ensure correct order

# Create stacked bar chart
fig, ax = plt.subplots(figsize=(7.5, 4))
crosstab.plot(kind='bar', stacked=True, ax=ax,
              color=['lightgray', 'lightblue', 'steelblue', 'darkblue', 'navy'],
              edgecolor='black', linewidth=0.5)

plt.xlabel('Communication Apprehension Quartile')
plt.ylabel('Proportion')
plt.title('Distribution of Transit Usage Categories by Communication Apprehension Quartile')
plt.legend(title='Transit Frequency', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.xticks(rotation=0)
plt.ylim(0, 1)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

Distribution of Transit Usage Categories by Communication Apprehension Quartile

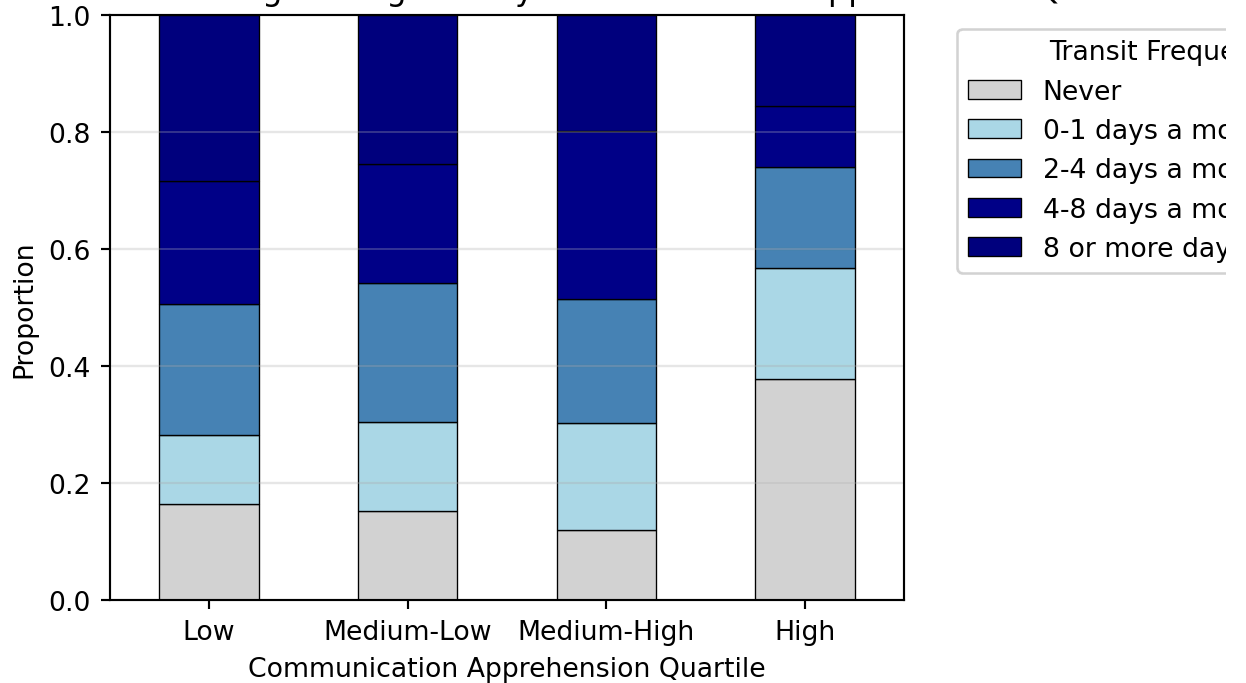


Figure 4: Proportional stacked bar chart of transit categories by communication apprehension quartiles

## Conclusion

The four visualization strategies evaluated in this memo each serve different analytical purposes and are suited to different forms of the dependent variable:

- Box plots ([Figure 1](#)) show distribution of communication apprehension scores by transit category but require sufficient sample size per category
- Jittered OLS regression ([Figure 2](#)) provides clear linear relationship information with confidence intervals but assumes unbounded continuous outcomes
- Logistic regression sigmoid curves ([Figure 3](#)) are theoretically appropriate for binary outcomes but discard frequency information
- Proportional stacked bar charts ([Figure 4](#)) preserve ordinal information and show distributional patterns but require discretization of continuous predictors

The choice of visualization should be guided by the specific research question, the nature of the forms of dependent variable, and the intended audience. For comprehensive analysis, researchers should consider presenting multiple complementary visualizations to fully capture the complexity of the relationship between communication apprehension and public transportation usage.